

Programación Básica en C



Patricio Olivares
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Herramientas a Utilizar

Durante este curso se trabajará con las siguientes herramientas para programar en C:

-  **Visual Studio Code (VS Code):** será el entorno de desarrollo (IDE) principal utilizado para editar, guardar y organizar programas en lenguaje C. Es una herramienta liviana y flexible, compatible con múltiples lenguajes de programación como C, Python y JavaScript. Puede descargarse desde el sitio oficial: <https://code.visualstudio.com>

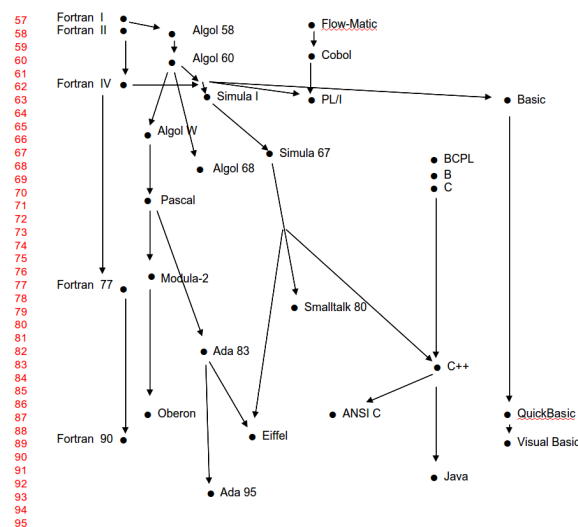
Extensiones recomendadas para trabajar en C:

- **C/C++ (Microsoft):** proporciona resaltado de sintaxis, autocompletado, navegación por el código y depuración.
- **Code Runner:** permite ejecutar fragmentos de código de manera rápida, útil para pruebas simples.
- **CMake Tools:** útil en proyectos más avanzados que utilicen CMake.

-  **Sistema operativo Linux:** todos los programas se compilarán y ejecutarán desde un entorno Linux, lo que permite trabajar directamente con herramientas de consola como `gcc` y tener un entorno estandarizado.

Importante: Aunque se utilizará un IDE como apoyo, esto no reemplaza el conocimiento de cómo compilar y ejecutar programas manualmente. A lo largo del curso se empleará el compilador `gcc` desde la terminal para entender los errores de compilación y ejecutar los programas paso a paso. Este enfoque permitirá desarrollar una comprensión más profunda del funcionamiento del lenguaje C.

Evolución de la programación



El Lenguaje C en la Historia de la Programación

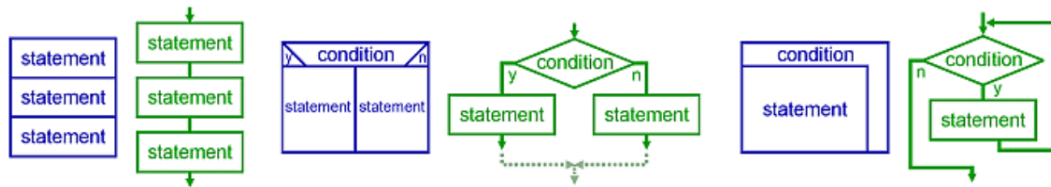
La imagen muestra una línea de tiempo con la evolución de los lenguajes de programación

- **El lenguaje C** aparece en los años 70 como una evolución de **B** y **BCPL**, ambos diseñados para facilitar la programación de sistemas.
- Su diseño se basa en conceptos heredados de lenguajes anteriores como **ALGOL**, pero con un enfoque más directo al hardware y al desarrollo de sistemas operativos.
- C fue la base para:
 - **ANSI C**, una versión estandarizada ampliamente adoptada.
 - **C++**, que agrega programación orientada a objetos.
 - **Java**, que tomó muchas ideas sintácticas de C/C++.

Programación Estructurada

Paradigma de programación basa en el **uso extensivo de subrutinas y estructuras de control** con el fin de mejorar la **claridad, calidad y tiempo de desarrollo** del código de programas computacionales.

- En contraste con saltos incondicionales como sentencia **goto** (difícil de seguir mantener)
- Partió con **ALGOL** en los '60



Introducción al Lenguaje C

C es un lenguaje de programación de propósito general, desarrollado por Dennis Ritchie en los años 70 en los laboratorios Bell. Es un lenguaje de programación de bajo nivel con características de alto nivel, lo que lo hace eficiente, poderoso y ampliamente utilizado para desarrollar sistemas operativos, controladores, y software embebido.

Se basa en el paradigma de programación estructurada y permite manipulación directa de memoria, lo cual lo convierte en una herramienta ideal para aprender fundamentos de programación.

¿Por qué aprender C?

- Ayuda a entender cómo funciona un computador a nivel interno (memoria, CPU, compilación).
- Refuerza conceptos fundamentales como tipos de datos, punteros y eficiencia del código.
- Sirve como base para aprender otros lenguajes como C++, Java y Rust.
- Es muy usado en contextos donde el rendimiento y control son críticos.

Ejemplos de uso del lenguaje C:

- El **núcleo de Linux** está escrito en C.
- Muchos microcontroladores, incluyendo **Arduino**, usan una variante de C/C++.
- Sistemas embebidos, drivers, firmware y herramientas de bajo nivel.

Características de C

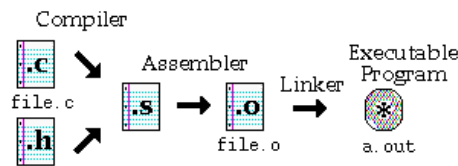
- **Eficiencia:** Permite acceso directo a memoria y manipulación de punteros.
- **Portabilidad:** Programas en C pueden compilarse en múltiples plataformas con pocos cambios.
- **Lenguaje procedural:** Se basa en funciones, con enfoque en el flujo de control.
- **Sintaxis simple y cercana al hardware.**
- **Amplio soporte:** Existe una gran comunidad y documentación.

El compilador gcc en Linux

`gcc` (GNU Compiler Collection) es el compilador estándar en sistemas Linux para el lenguaje C. Fue desarrollado por el proyecto GNU y es ampliamente utilizado debido a su eficiencia, flexibilidad y compatibilidad con estándares de programación.

En el contexto del lenguaje C, `gcc` se encarga de:

- Preprocesar (`#include` , `#define`)
- Compilar el código fuente (`.c`) a código objeto (`.o`)
- Enlazar los objetos para generar un ejecutable final



Fuente: <http://profesores.elo.utfsm.cl/~agv/elo320/makefile/makefile.html>

Para poder utilizar el compilador de C en su entorno Linux, debe asegurarse que este se encuentre instalado. Puede verificarlo ejecutando en su terminal:

```
gcc --version
```

Si este no se encuentra instalado, puede realizar la instalación de manera manual. En Ubuntu (y derivados), esto se realiza con el siguiente comando:

```
sudo apt install build-essential
```

¿Cómo compilar un programa en C?

Preprocesamiento y compilación

```
gcc -c mi_programa.c -o mi_programa.o
```

Enlazado (se pueden enlazar múltiples archivos .o)

```
gcc mi_programa.o -o mi_programa
```

```
# Ejecución
./mi_programa
```

Python vs C

Aunque Python y C comparten conceptos como variables, operadores y funciones, difieren en su nivel de abstracción. Python es más fácil de leer y escribir, ideal para empezar a programar. C, en cambio, ofrece mayor control del hardware, exige declarar tipos de datos y requiere compilar el código antes de ejecutarlo.

Característica	Python	C
Nivel de abstracción	Alto (lenguaje interpretado, fácil de leer)	Bajo (lenguaje compilado, más cercano al hardware)
Tipado	Dinámico (no se declara el tipo)	Estático (debes declarar el tipo de cada variable)
Compilación	No necesita compilar, se ejecuta directamente	Requiere compilación previa (gcc archivo.c)
Gestión de memoria	Automática (recolección de basura)	Manual (el programador gestiona la memoria)
Sintaxis	Simple, indentación obligatoria	Más verboso, usa {} y ;
Velocidad de ejecución	Más lento (interpretado)	Más rápido (compilado y optimizado)

Python es ideal para comenzar a programar por su simplicidad, mientras que C ofrece más control sobre cómo funciona el programa internamente.

Primer programa en C

```
#include <stdio.h>           // Incluye la biblioteca estándar para
                               entrada/salida

int main() {                  // Función principal: punto de inicio del
                               programa
    printf("Hola mundo en C\n"); // Imprime un mensaje en pantalla
    return 0;                  // Retorna 0 al sistema operativo (indica
                               ejecución correcta)
}
```

Explicación del primer programa en C

Este programa realiza una de las tareas más comunes al iniciar en un lenguaje: imprimir "Hola mundo" en pantalla. A continuación se explican sus componentes:

- `#include <stdio.h>` : Importa la biblioteca estándar de entrada y salida, necesaria para usar funciones como `printf`.
- `int main()` : Define la función principal del programa. Es el punto donde inicia la ejecución.
- `printf(...)` : Imprime texto en la consola. El `\n` representa un salto de línea.
- `return 0;` : Finaliza el programa y devuelve 0 al sistema operativo, indicando que terminó correctamente.

Este es un ejemplo básico, pero representa la estructura típica de un programa en C: inclusión de bibliotecas, definición de `main()`, instrucciones y retorno.

Usaremos como base un programa simple en C como este:

```
#include <stdio.h>

int main() {
    printf("Hola mundo\n");
    return 0;
}
```

Comparación con Python: Versión básica

El siguiente código en Python es equivalente al programa en C anterior. Ambos imprimen un mensaje en consola:

```
print("Hola mundo")
```

Similitudes:

- Ambos programas imprimen texto en pantalla.
- Son ejecutables sin argumentos ni estructuras de control adicionales.

Diferencias:

- En C, se necesita una función principal (`main`) y el uso de `#include <stdio.h>` para usar `printf`.
- En Python, basta con una línea usando `print()`.

Comparación con Python: Versión extendida

Una forma más estructurada y cercana al estilo de C en Python es encapsular la lógica en una función `main` y usar el bloque `if __name__ == "__main__":`:

```
def main():
    print("Hola mundo")

if __name__ == "__main__":
    main()
```

Similitudes con C:

- Hay una función principal donde se ejecuta el código (`main`).
- El bloque `if __name__ == "__main__"` garantiza que el código solo se ejecute si el archivo se corre directamente (comparable a `main()` en C).

Diferencias:

- En C, `main` es obligatorio. En Python, es opcional pero buena práctica.
- En C se usa `return 0;` para indicar que el programa terminó bien. En Python no es necesario (aunque se puede usar `sys.exit(0)` si se desea).

Comentarios en C

Los comentarios son útiles para documentar el código y hacerlo más fácil de entender.

- **Comentario de una línea:**

```
// Este es un comentario de una sola línea
```

- **Comentario de múltiples líneas:**

```
/*
Este es un comentario
que ocupa más de una línea
*/
```

Variables y Tipos de Datos

Las variables almacenan información que puede cambiar durante la ejecución del programa.

Ejemplos de tipos comunes:

- `int` : números enteros
- `float` : números decimales de menor precisión
- `double` : números decimales de mayor precisión
- `char` : un carácter

```
int edad = 20;
float altura = 1.75;
char inicial = 'P';
```

Entrada y Salida estándar

En C usamos las funciones `scanf` y `printf` para interactuar con el usuario a través de la consola. Estas funciones pertenecen a la biblioteca estándar `<stdio.h>`.

```
#include <stdio.h>
```

printf

`printf` se utiliza para **mostrar información por pantalla**. Puede imprimir texto, variables y resultados.

Ejemplo básico:

```
int edad = 20;
printf("Tu edad es: %d\n", edad);
```

- El texto se escribe tal cual (entre comillas dobles).
- El símbolo `%d` es un **formato de impresión** que indica dónde se mostrará el valor de `edad`.
- `\n` representa un salto de línea.

scanf

`scanf` se utiliza para **leer datos desde el teclado** y guardarlos en variables.

Ejemplo:

```
int edad;
printf("Ingresa tu edad: ");
scanf("%d", &edad);
```

- `%d` indica que se espera un número entero.
- `&edad` es la dirección de memoria donde se almacenará el valor ingresado.
- Es obligatorio usar `&` (ampersand) para pasar la **dirección de la variable**, salvo con arreglos de caracteres (`char[]`), donde no es necesario.

Códigos de formato comunes

Código	Significado	Tipo de dato en C
<code>%d</code>	Entero decimal (%d)	<code>int</code>
<code>%f</code>	Número decimal (float)	<code>float</code> , <code>double</code>
<code>%lf</code>	Decimal de doble precisión	<code>double</code>
<code>%c</code>	Carácter individual	<code>char</code>
<code>%s</code>	Cadena de caracteres	<code>char[]</code> (arreglo)

Constantes y Macros

Para definir valores que no cambian, se pueden usar:

1. Macros con `#define` :

```
#define PI 3.1416
```

2. Constantes con `const` :

```
const float PI = 3.1416;
```

Diferencias:

- `#define` es manejado por el preprocesador (sin tipo).
- `const` respeta los tipos y permite depuración más clara.

Comparación: Operadores en C vs Python

Tanto **C** como **Python** comparten muchos operadores con funciones similares, pero la **sintaxis puede variar ligeramente**. A continuación se muestra una tabla comparativa con los más usados:

Tipo de operador	C	Python	Comentario
-	Aritméticos <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> Coinciden en comportamiento básico	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> <code>x / y</code> (entero si <code>int</code>) <code>x // y</code>	
	En C depende del tipo de dato	Potencia <code>pow(x, y)</code> (en <code><math.h></code>) <code>x ** y</code> C no tiene operador <code>**</code>	
	Relacionales <code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> <code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> Comparaciones directas	Lógicos <code>&&</code> , <code>\ \ </code> , <code>!</code> <code>and</code> , <code>or</code> , <code>not</code> Sintaxis diferente pero mismo propósito	
	Asignación compuesta <code>+=</code> , <code>-=</code> , <code>*=</code> , etc. <code>+=</code> , <code>-=</code> , <code>*=</code> , etc. Mismos operadores		
	Booleanos <code>1</code> (true), <code>0</code> (false) <code>True</code> , <code>False</code> Tipos booleanos explícitos en Python		

Ejemplos comparativos: Python vs C

Ejemplo 1: Declarar y sumar variables

Python

```
a = 2
b = 4
suma = a + b
print("Resultado:", suma)
```

C

```
#include <stdio.h>

int main() {
    int a = 2, b = 4;
```

```

    int suma = a + b;
    printf("Resultado: %d\n", suma);
    return 0;
}

```

Ejemplo 2: Leer dos enteros y mostrar la suma

Python

```

a = int(input("Ingresa un número: "))
b = int(input("Ingresa otro número: "))
print("La suma es:", a + b)

```

C

```

#include <stdio.h>

```

```

int main() {
    int a, b;
    printf("Ingresa un número: ");
    scanf("%d", &a);
    printf("Ingresa otro número: ");
    scanf("%d", &b);

    int suma = a + b;
    printf("La suma es: %d\n", suma);
    return 0;
}

```

Ejemplo 3: Leer una letra

Python

```

letra = input("Ingresa una letra: ")
print("Ingresaste:", letra)

```

C

```

#include <stdio.h>

```

```

int main() {
    char letra;
    printf("Ingresa una letra: ");
    scanf(" %c", &letra); // Espacio antes de %c limpia el buffer
    printf("Ingresaste: %c\n", letra);
    return 0;
}

```

Ejemplo 4: Uso de constantes

Python

```

PI = 3.1416
radio = 5
area = PI * radio * radio
print("Área:", area)
C

#include <stdio.h>

#define PI 3.1416

int main() {
    int radio = 5;
    float area = PI * radio * radio;
    printf("Área: %.2f\n", area);
    return 0;
}

```

Estructuras de Control

1. if, if-else, if-else if-else

Uso: Ejecuta un bloque de código si se cumple una condición; con `else` y `else if` se pueden definir caminos alternativos. **Sintaxis:**

```

if (condicion) {
    // código si verdadero
} else if (otra_condicion) {
    // código si segunda condición es verdadera
} else {
    // código si ninguna condición es verdadera
}

```

Ejemplo:

```

#include <stdio.h>

int main(void) {
    int x;
    printf("Ingrese un número: ");
    scanf("%d", &x);

    if (x < 0) {
        printf("Negativo\n");
    } else if (x == 0) {
        printf("Cero\n");
    } else {
        printf("Positivo\n");
    }
    return 0;
}

```

2. Cortocircuito en && y ||

Uso: Combina condiciones; && exige que ambas sean verdaderas, || con que una lo sea. **Sintaxis:**

```
if (cond1 && cond2) { /* ... */ }
if (cond1 || cond2) { /* ... */ }
```

La evaluación se detiene si ya se conoce el resultado.

Ejemplo:

```
#include <stdio.h>

int main(void) {
    int a = 0, b = 5;

    if (a != 0 && (++b > 5)) { // No evalúa la derecha si la
        // izquierda es falsa
        printf("Se actualizó b\n");
    }
    printf("b = %d\n", b);
    return 0;
}
```

3. switch-case

Uso: Selecciona un bloque de código entre varias opciones, comparando el valor de una variable con constantes. **Sintaxis:**

```
switch (variable) {
    case valor1:
        // código
        break;
    case valor2:
        // código
        break;
    default:
        // código si no hay coincidencia
}
```

Ejemplo:

```
#include <stdio.h>

int main(void) {
    int opcion;
    printf("Ingrese opción (1-3): ");
    scanf("%d", &opcion);

    switch (opcion) {
```

```

        case 1: printf("Opción 1\n"); break;
        case 2: printf("Opción 2\n"); break;
        case 3: printf("Opción 3\n"); break;
        default: printf("Opción no válida\n");
    }
    return 0;
}

```

4. Bucle `while`

Uso: Repite un bloque mientras la condición sea verdadera. **Sintaxis:**

```

while (condicion) {
    // código
}

```

Ejemplo:

```

#include <stdio.h>

int main(void) {
    int i = 1;
    while (i <= 5) {
        printf("%d ", i);
        i++;
    }
    printf("\n");
    return 0;
}

```

5. Bucle `do-while`

Uso: Igual que `while`, pero garantiza al menos una ejecución antes de evaluar la condición. **Sintaxis:**

```

do {
    // código
} while (condicion);

```

Ejemplo:

```

#include <stdio.h>

int main(void) {
    int n;
    do {
        printf("Ingrese un número positivo: ");
        scanf("%d", &n);
    } while (n <= 0);
    printf("Número aceptado: %d\n", n);
}

```

```
    return 0;
}
```

6. Bucle for

Uso: Ideal para repeticiones con contador conocido. **Sintaxis:**

```
for (inicializacion; condicion; actualizacion) {
    // código
}
```

Ejemplo:

```
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 5; i++) {
        printf("i = %d\n", i);
    }
    return 0;
}
```

Funciones y Procedimientos

Funciones en C

¿Qué es una función y para qué sirve?

Una **función** es un bloque de código con un nombre, que realiza una tarea específica y que puede reutilizarse en distintas partes del programa. Las funciones ayudan a:

- **Modularizar** el código, dividiéndolo en partes más pequeñas y manejables.
- **Reutilizar** código sin tener que escribirlo varias veces.
- Mejorar la **legibilidad** y mantenimiento del programa.

Sintaxis general

Prototipo (declaración):

```
tipo_retorno nombre_funcion(tipo1 parametro1, tipo2 parametro2, ...);
```

Definición:

```
tipo_retorno nombre_funcion(tipo1 parametro1, tipo2 parametro2, ...) {
    // código de la función
    return valor; // si tipo_retorno no es void
}
```

Llamada:

```
resultado = nombre_funcion(arg1, arg2);
```

Separación en archivos `.h` y `.c`

En C es buena práctica **separar**:

- **Archivo de encabezado (`.h`)**: contiene prototipos de funciones, macros y definiciones necesarias para que otros archivos puedan usar la función.
- **Archivo de implementación (`.c`)**: contiene el código (definiciones) de las funciones declaradas en el `.h`.
- **Archivo principal (`main.c`)**: programa principal que usa las funciones.

Diferencia entre `#include <>` y `#include " "`

- `#include <archivo>` : busca el archivo **solo** en las rutas estándar del compilador (bibliotecas del sistema).
- `#include "archivo"` : busca primero en el directorio del proyecto y, si no lo encuentra, en las rutas estándar. Ejemplo:

```
#include <stdio.h>    // biblioteca estándar
#include "funciones.h" // archivo propio del proyecto
```

`funciones.h`

```
// funciones.h
/* Devuelve 1 si n es par, 0 en otro caso. */
int es_par(int n);

/* Imprime un saludo simple. */
void saludar(void);
```

`funciones.c`

```
// funciones.c
#include "funciones.h"
#include <stdio.h>

int es_par(int n) {
    return (n % 2 == 0);
}

void saludar(void) {
    printf("Hola TEL102!\n");
}
```

main.c

```
// main.c
#include <stdio.h>
#include "funciones.h"

int main(void) {
    saludar();
    int x = 7;
    if (es_par(x)) {
        printf("%d es par\n", x);
    } else {
        printf("%d es impar\n", x);
    }
    return 0;
}
```

Compilación

En un solo paso:

```
gcc -o app main.c funciones.c
./app
```

En pasos separados:

```
gcc -c main.c          # genera main.o
gcc -c funciones.c     # genera funciones.o
gcc -o app main.o funciones.o
./app
```

Arreglos en C

¿Qué es un arreglo?

Un **arreglo** es una colección de elementos del mismo tipo, almacenados de forma contigua en memoria y accesibles mediante un índice numérico. En C:

- El índice empieza en **0**.
- El tamaño debe ser conocido en tiempo de compilación (a menos que se use memoria dinámica).

1) Declaración e inicialización

Sintaxis:

```
tipo nombre[tamaño];
tipo nombre[tamaño] = {valor1, valor2, ...};
```


Ejemplo:

```
#include <stdio.h>

int main(void) {
    int numeros[5] = {1, 2, 3, 4, 5};
    for (int i = 0; i < 5; i++) {
        printf("%d ", numeros[i]);
    }
    printf("\n");
    return 0;
}
```

2) Recorrido con for

```
#include <stdio.h>

int main(void) {
    int datos[3];
    for (int i = 0; i < 3; i++) {
        printf("Ingrese valor %d: ", i);
        scanf("%d", &datos[i]);
    }
    printf("Valores: ");
    for (int i = 0; i < 3; i++) {
        printf("%d ", datos[i]);
    }
    printf("\n");
    return 0;
}
```

3) Paso de arreglos a funciones

Los arreglos pueden ser pasados como entrada a función.

```
#include <stdio.h>

void imprimir(int arr[], int tam) {
    for (int i = 0; i < tam; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main(void) {
    int numeros[4] = {10, 20, 30, 40};
    imprimir(numeros, 4);
    return 0;
}
```

4) Cadenas de caracteres

En C, una cadena es un arreglo de `char` terminado con `'\0'`.

```
#include <stdio.h>
```

```
int main(void) {  
    char nombre[10] = "USM";  
    printf("Nombre: %s\n", nombre);  
    return 0;  
}
```

5) Uso de la biblioteca `<string.h>`

C provee la biblioteca estándar `<string.h>` para trabajar con arreglos de caracteres.

Algunas funciones comunes son:

- `strlen(cadena)` → devuelve la longitud (sin contar `'\0'`).
- `strcpy(destino, origen)` → copia una cadena.
- `strcat(destino, origen)` → concatena al final de otra cadena.
- `strcmp(cad1, cad2)` → compara cadenas (0 si son iguales).

Ejemplo:

```
#include <stdio.h>  
#include <string.h>
```

```
int main(void) {  
    char saludo[20] = "Hola";  
    char nombre[] = "Mundo";  
  
    strcat(saludo, " ");    // concatena espacio  
    strcat(saludo, nombre); // concatena "Mundo"  
    printf("%s\n", saludo); // Hola Mundo  
    printf("Longitud: %lu\n", strlen(saludo));  
    return 0;  
}
```

Estructuras en C

¿Qué es una estructura?

Una **estructura** en C es un tipo de dato compuesto que permite agrupar variables de **diferentes tipos** bajo un mismo nombre. Se usa para representar entidades u objetos con múltiples atributos.

Sintaxis

```

struct NombreEstructura {
    tipo campo1;
    tipo campo2;
    // más campos...
};

```

Para declarar variables de ese tipo:

```

struct NombreEstructura var1, var2;

```

Ejemplo

```

#include <stdio.h>
#include <string.h>

struct Persona {
    char nombre[30];
    int edad;
};

int main(void) {
    struct Persona p1;

    strcpy(p1.nombre, "Ana");
    p1.edad = 25;

    printf("Nombre: %s\n", p1.nombre);
    printf("Edad: %d\n", p1.edad);

    return 0;
}

```

Ejericicios

Ejercicio 1: Traducción de Python a C

Traduce el siguiente código Python al lenguaje C, usando `printf`, `scanf`, operadores aritméticos y declaración de variables:

```

# Calculadora de área de rectángulo
base = float(input("Ingresa la base: "))
altura = float(input("Ingresa la altura: "))
area = base * altura
print("El área es:", area)

```

Indicaciones:

- Usa variables tipo `float` o `double`.
- Usa `scanf` para leer los datos.
- Usa `printf` para mostrar el resultado con dos decimales.

- Agrega comentarios al código en C.

Ejercicio 2: Programa simple en C

Escribe un programa en C que:

1. Declare una constante `#define` llamada `IVA` con valor `0.19`.
2. Pida al usuario ingresar un precio base.
3. Calcule el precio final con IVA.
4. Muestre el resultado por pantalla.

Ejemplo de salida esperada:

Ingresa el precio base: 10000

Precio final con IVA: 11900.00

Recuerda: Usa `float`, `#define`, `scanf`, `printf` y operadores aritméticos.

Ejercicio 3: Uso de estructuras de control

Crea un programa en C que:

1. Pida al usuario un número entero del 1 al 3.
2. Use un `switch` para:
 - Si es `1`, imprimir "Hola".
 - Si es `2`, imprimir los números del 1 al 5 usando un `for`.
 - Si es `3`, pedir un número y verificar con `if` si es par o impar.
 - Si es otro valor, mostrar "Opción no válida".
3. El programa debe repetirse hasta que el usuario ingrese `0` (usar `while` o `do-while`).

Ejercicio 4: Uso de funciones

Crea un programa en C que incluya una función para determinar el mayor de dos enteros, separando **declaración** (`.h`) e **implementación** (`.c`).

1. `util.h`
 - Declara la función `max2(int a, int b)`.
2. `util.c`
 - Incluye `"util.h"`.
 - Implementa `max2` para que retorne el mayor de los dos enteros.
3. `main.c`

- Incluye `<stdio.h>` y `"util.h"`.
- Pide al usuario dos enteros, llama a `max2` y muestra el resultado.

4. **Compila y ejecuta:**

Ejemplo de salida:

Ingrese a y b: 7 12
max(7, 12) = 12

Ejercicio 5: Uso de arreglos

Crea un programa que:

1. Declare un arreglo de 5 enteros.
2. Pida al usuario que ingrese los 5 valores.
3. Calcule y muestre la **suma total** de los elementos.
4. Muestre los valores ingresados en orden inverso.

Ejercicio 6: Uso de estructuras

Crea un programa en C que cumpla con los siguientes requisitos:

1. Define una estructura llamada `Producto` que contenga:
 - Un `char` arreglo de 30 caracteres para el nombre.
 - Un `float` para el precio.
 - Un `int` para la cantidad en stock.
2. En el `main`, declara una variable de tipo `Producto`.
3. Solicita al usuario los datos para esa variable.
4. Muestra por pantalla la información ingresada en un formato claro.

Ejemplo de ejecución esperada:

Ingrese nombre del producto: Café
Ingrese precio: 2.5
Ingrese cantidad en stock: 10
Producto: Café
Precio: \$2.50
Cantidad en stock: 10